AI-Powered Fire Prediction Spread and Containment System

Software Architecture Specifications

Panic! At The Kernel
Client: EPI-USE Africa

October 2026



Full Name	Student Number
Megan Lai	u23608821
Inge Keyser	u23544563
Janri du Toit	u23632730
Ryan Lynn	u23541912
Marco de Wit	u22811975

Contents

1 Introduction **4**

2 Architectural Requirements **4**
2.1 Architectural Patterns . . . 4
2.1.1 3-Layer Architecture . . . 4
2.1.2 Queue . . . 4
2.1.3 Circuit Breaker Pattern . . . 4
2.2 Architectural Diagram . . . 5
2.3 Design Patterns . . . 6
2.3.1 Singleton . . . 6
2.3.2 Adapter . . . 6
2.3.3 Observer . . . 6
2.3.4 State . . . 6
2.3.5 Command . . . 7
2.4 Constraints . . . 7
2.4.1 Resources From The Client . . . 7
2.4.2 Real-Time Processing . . . 8
2.4.3 Third-Party Dependency . . . 8
2.4.4 Data Currency . . . 8
2.4.5 Shared Data Access . . . 8
2.5 Mapping Quality Requirements to Architectural Decisions . . . 9

3 Technology Requirements **10**
3.1 Technology Requirements Choices . . . 10
3.1.1 Frontend . . . 10
3.1.2 Backend . . . 10
3.1.3 Mapping . . . 11
3.1.4 AI/ML . . . 11
3.1.5 Database . . . 11
3.1.6 Cache . . . 11
3.1.7 Notifications . . . 11
3.1.8 Versioning . . . 11
3.1.9 Containerization . . . 12
3.1.10 Deployment . . . 12
3.1.11 Testing . . . 12

4 API Contracts **12**
4.1 Authentication & Access Control Service . . . 12
4.2 Fire Reporting & Verification Service . . . 13
4.3 AI Fire Spread Simulation Engine . . . 13
4.4 Live Fire Map Service . . . 14
4.5 Proximity Alert Service . . . 14
4.6 Firefighter Tactical Service . . . 14

5 Deployment **15**
5.1 Deployment Requirements . . . 15
5.1.1 Live, Accessible System . . . 15
5.1.2 Environment Parity . . . 15
5.1.3 Rollback Strategy . . . 15
5.2 Deployment Diagram . . . 16
5.3 CI/CD Pipeline Diagram . . . 17

6 Change Log **18**

1 Introduction

This Software Architecture Specifications document is for the AI-Powered Fire Spread Prediction and Containment System, Fire Away, and specifies how the system of this application is structured to satisfy the Systems Requirements Specifications.

2 Architectural Requirements

2.1 Architectural Patterns

2.1.1 3-Layer Architecture

The Presentation Layer, Business Logic Layer, and Data Layer make up the 3 Layers of the layered architecture. The presentation layer holds the user interface and request handling, which check who the user is and pass their requests on. The business layer holds the rules for reporting, verification, prediction and alerts, and the data layer handles the storage, caching and queuing of information.

2.1.2 Queue

A job queue is used so that heavy fire simulations run in the background without slowing down the web server. A repeat request is answered straight from a cache. Otherwise the job goes to a free volunteer GPU machine, or to a waiting queue that a GPU worker clears one job at a time. A failed job is automatically retried, and a request that gets no result within a set time returns a timeout error.

2.1.3 Circuit Breaker Pattern

Fire Away wraps every call to an outside service, such as weather, maps, email and the volunteer GPU machines, in a circuit breaker. After repeated failures the breaker opens and the app stops calling the failing service. Instead it uses a fallback, such as a cached result, a skipped check or the job queue. After a cooldown it lets one trial call through, and it closes again only if that call succeeds. This keeps one failing service from slowing down or breaking the rest of the system.

2.2 Architectural Diagram

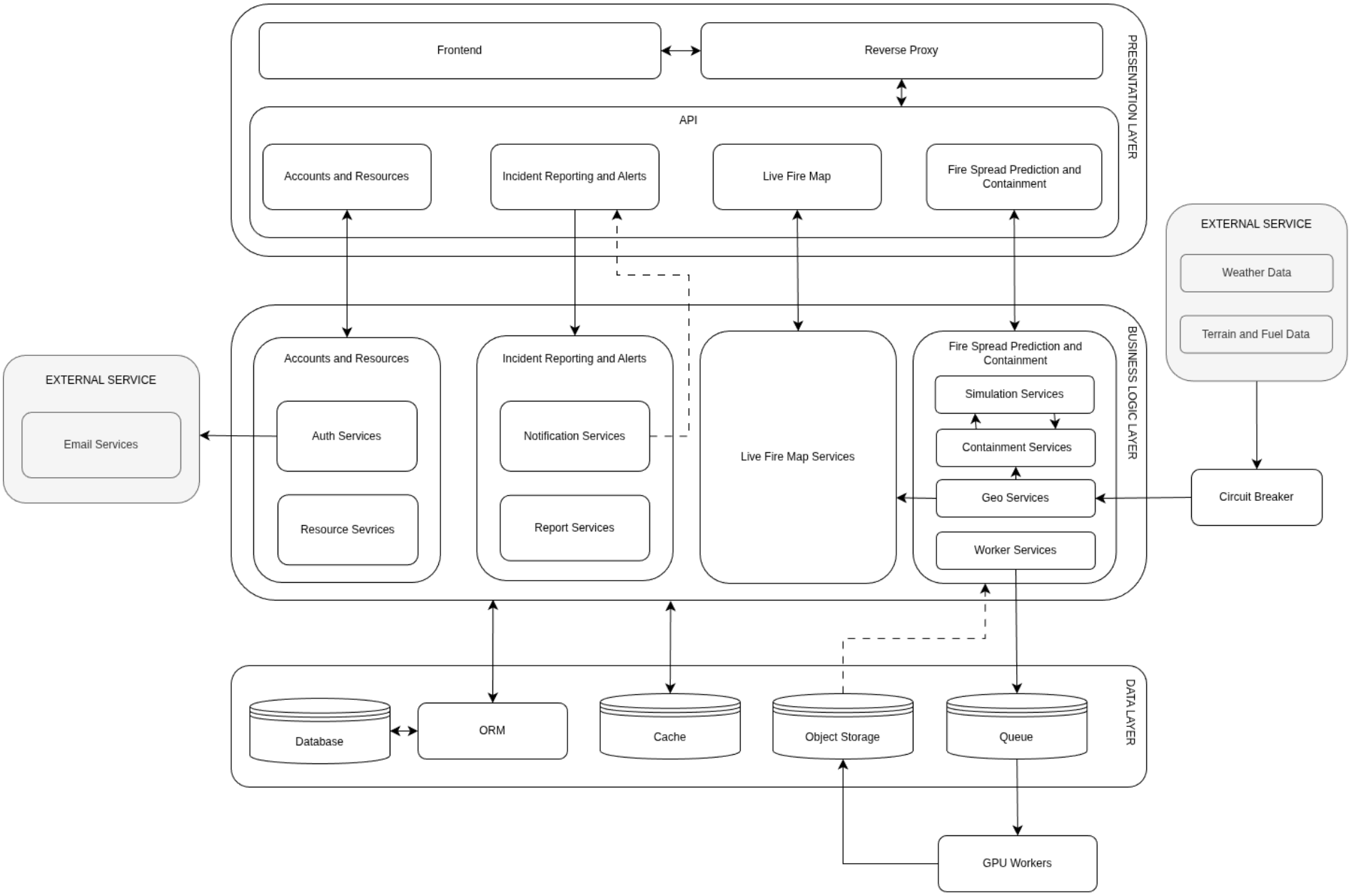


Figure 1: Architecture Diagram

2.3 Design Patterns

2.3.1 Singleton

We use a singleton for the database engine and the session factory. The singleton pattern enforces that only a single instance of an object exists. Having only one instance of the database engine and session factory ensures that the application only has one source of truth and cannot respond with outdated or incorrect data across concurrent requests.

2.3.2 Adapter

Used to convert external data into the format the fire spread model requires. The model expects fixed-size grids of floating-point values, whereas the system receives weather data as a JSON response from an external service and terrain and vegetation data as raster files. A set of adapter functions in the simulation job builder sits between these sources and the model, converting wind speed and direction into directional components and resampling terrain and fuel data to the model's grid shape. Because the adapter is implemented as functions rather than a dedicated class, it is a lightweight, functional form of the pattern, but serves the same purpose of decoupling the model from the formats of its external data sources.

2.3.3 Observer

Live alerts are pushed to users' browsers through a central registry of open real-time connections. The registry records which connections belong to which user. A user can have several open at once for example in two browser tabs. It adds a connection when a client connects and removes it when the client disconnects, so the list always matches who is online. A lock protects these changes so simultaneous connections and disconnections cannot corrupt the list. When an alert is raised, the system looks up the intended recipient(s) and delivers the message to every connection they have open. A message to several recipients is sent to all of them at the same time. Connections that fail during delivery are removed afterwards. If a recipient has no open connection, the real-time push is skipped. The alert is also stored so the user sees it the next time they load the application. This follows the idea of interested parties registering and being notified when something happens. It differs from the textbook definition because subscriptions are tied to users not to event types. The registry is also held in memory within a single backend process.

2.3.4 State

Entities that move through a lifecycle, such as fire reports and volunteer GPU machines, are modeled with a fixed set of named statuses. This keeps the allowed states in one place, and the database, API and user interface use the same names. A report moves through received, pending and then one of two final outcomes, verified or rejected. Each status has a numeric rank with the two final outcomes sharing the highest rank. This makes the order explicit, so the system can check that a report only moves forward and never backwards. A volunteer machine has its own set of statuses:

- **Active:** Connected, healthy, and ready for work.
- **Busy:** Running a simulation
- **Offline:** Lost contact
- **Rejected:** Failed the hardware check at at registration.
- **Quarantined:** Temporarily held back for 15 minutes after a failure or timeout
- **Deactivated:** Paused by the volunteer

- **Removed:** Revoked by an admin

Each status is also tied to a colour on the dashboard, so operators can read the state of the pool at a glance. This is a lightweight form of the state pattern. The code that reacts to events such as a connection, a job queue being dispatched or a missed heartbeat enforces which transitions are allowed.

2.3.5 Command

Each simulation run is packaged as a self-contained job message, so the part of the system that creates the job doesn't need to know who will run it. Everything the simulation needs travels inside inside the message:

- A unique job identifier
- Recent weather history
- Terrain and vegetation grids
- Location where the fire starts
- Grid size and bounds
- Number of time steps to simulate
- Any containment lines
- The model parameters

For volunteer machines, the grid data is converted into a plain transferrable format so the whole message can be sent over a real-time connection. The same job can instead be placed on the processing queue for the dedicated cloud GPU worker. Either way, the result comes back tagged with the same job identifier. Because the message carries all of its inputs, it can be sent to whichever machine is available, queued, retried after a timeout, or re-sent to a different machine after a failure. It does not depend on any shared state between the requester and the executor. This matches the command pattern's purpose of turning a request into an object that can be passed around, queued and run later.

2.4 Constraints

2.4.1 Resources From The Client

Data Sets recommended but not limited to the following:

- **Fire Occurrence & Historical Fires**
  - NASA Global Fire Atlas (2003–2016) – Fire dynamics, duration, spread patterns: https://data.nasa.gov/dataset/global-fireatlas-with-characteristics-of-individual-fires-2003-2016-69d65
  - NASA FIRMS Active Fire Data (MODIS & VIIRS) – Near real-time fire hotspots: https://firms.modaps.eosdis.nasa.gov/map/#d:today;@28.388,-25.445,16.000z
  - AFIS South African Fire Data – Regional fire data: https://www.afis.co.za/data/
  - Research Fire Dataset (arXiv reference) – Dataset and methodology reference: https://arxiv.org/abs/2412.11555
- **Infrastructure & Human Settlements**

  - OpenStreetMap (OSM) – Roads, buildings, rivers, power lines: https://www.openstreetmap.org/

- **Weather & Fire Weather Index**
  - ERA5 Reanalysis (ECMWF) – Global historical and real-time weather: wind, temperature, humidity, precipitation: https://cds.climate.copernicus.eu/cdsapp#!/dataset/reanalysis-era5-single-levels
  - South African Weather Service (SAWS) – Local weather feeds for real-time predictions: https://www.weathersa.co.za/

- **Vegetation & Fuel Load**
  - MODIS Vegetation Indices (NDVI/EVI) – Vegetation greenness for fuel condition: https://earthdata.nasa.gov/
  - SMAP Soil Moisture Dataset (NASA) – Soil moisture to estimate dryness: https://nsidc.org/data/smap
  - Land Cover Data (ESA WorldCover / Copernicus) – Fuel type classification: https://esa-worldcover.org/

- **Terrain / Topography**
  - ASTER GDEM – Alternative elevation dataset: https://asterweb.jpl.nasa.gov/gdem.asp

2.4.2 Real-Time Processing

Fire spread predictions must be generated quickly enough to remain useful during an active incident, placing a hard limit on the processing time available for terrain and spread calculations.

2.4.3 Third-Party Dependency

The system depends on an external weather data provider that it does not control, hence it is exposed to that provider's rate limits, latency, and potential downtime.

2.4.4 Data Currency

When the external weather source is unavailable, the system must continue operating using previously cached data, which means predictions may temporarily rely on information that is no longer current.

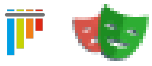
2.4.5 Shared Data Access

Multiple components that read and write fire-prediction data need access to a shared, up-to-date data source, which constrains how independently those components can scale or operate without risk of data conflicts or contention.

2.5 Mapping Quality Requirements to Architectural Decisions

Quality Requirement	**Architectural Decision**
Scalability (Support 1000+ concurrent users)	Horizontal scaling mechanisms within the distributed system layers.
Security (AES-256, RBAC, API auth)	Security Proxy for traffic validation and authentication controller.
Availability (99.9% uptime, local caching)	Layered architecture with local client-side caching and resillient data synchronization.
Performance (3s prediction, 95% of simulation requests rendered)	Blackboard architecture enabling collaborative, asynchronous data processing.
Reliability (Recover within 2 mins, handle input loss)	Circuit Breaker pattern on Weather Data Adapter to manage external service failures.
Maintainability (Integrate new data/fire risk models in <12 hours)	Adapter pattern and modular Service Layer design isolating external dependencies.

3 Technology Requirements

Layer	Technology	
Frontend	React, Next.js, DaisyUI	
Backend	Python, FastAPI	
Mapping	Mapbox GL	
AI/ML	PyTorch	
Database	PostgreSQL, PostGIS, SQLAlchemy	
Cache	Valkey	
Notifications	pywebpush	
Versioning	Git, GitFlow	
Containerization	Docker	
Deployment	Amazon Web Services	
Testing	Pytest, Playwright	

3.1 Technology Requirements Choices

3.1.1 Frontend

- React
- Next.js
- DaisyUI

React's concurrent rendering handles simultaneous live map updates without bottlenecks. Next.js SSR ensures WCAG 2.1 compliance as screen readers require fully rendered HTML. DaisyUI handles the structural baseline and the state logic of UI elements, allowing to treat the frontend as a modular, plug-and-play system.

3.1.2 Backend

- Python
- FastAPI

Python has mature support for the full AI/ML stack required (PyTorch, scikit-learn, GeoPandas, GDAL). FastAPI's async model concurrently ingests live ERA5, SAWS, and NASA FIRMS data without blocking, and runs at 21,000+ requests/second (above the 1,000 concurrent user requirement). Pydantic validation meets the 200ms form validation delivery requirement at the API layer.

3.1.3 Mapping

- Mapbox GL

GPU-accelerated WebGL rendering redraws fire spread polygons and overlays in under 16ms. Built-in offline tile packs allow firefighters to use the map without internet. The library is open source and free, with a 50000 map loads/month free tier.

3.1.4 AI/ML

- PyTorch

PyTorch's CNN achieves F1 scores of 0.64 - 0.72 on fire spread benchmarks. ts LSTM layer handles the 30-day vegetation dryness and multi-hour wind forecasts required by the 24 - 72hr risk engine. The autograd engine enables post-event model fine-tuning.

3.1.5 Database

- PostgreSQL
- PostGIS
- SQLAlchemy

PostGIS provides 1 000+ geospatial functions for cross-referencing fire paths with OSM roads and settlements. ACID transactions guarantee complete audit records with rollback capability, ensuring firefighter action logging. GiST spatial indexes enable sub-second proximity queries powering automated infrastructure alerts. SQLAlchemy provides a secure, programmatic database abstraction layer that natively integrates with spatial extensions to handle complex geospatial coordinate data, like fire pins and containment lines, using clean Python code. Additionally, its built-in connection pooling and automated parameter sanitization safeguard your platform against SQL injection while maintaining high performance during sudden concurrent traffic spikes.

3.1.6 Cache

- Valkey

Valkey operates entirely in-memory, delivering sub-millisecond read times. When the AI simulation engine needs wind, humidity, and vegetation data instantly, pulling it from Valkey rather than making repeated external API calls or querying a disk-based SQL database ensures the prediction grid is generated within seconds.

3.1.7 Notifications

- pywebpush

Pywebpush implements the standard browser Web Push protocol, including message encryption and VAPID signing.

3.1.8 Versioning

- Git
- GitFlow

Git paired with Gitflow provides isolated, risk-free environments to develop complex features like the AI simulation engine without compromising the stability of the live, safety-critical application. It establishes a dedicated pipeline for rigorous testing and rapid emergency hotfixes, ensuring the platform remains highly reliable and continuously available for field operations.

3.1.9 Containerization

- Docker

Docker containerization locks down all complex geospatial and AI dependencies into an identical environment across development and production, eliminating configuration errors and safeguarding system reliability. It enables lightweight, isolated services that can spin up in milliseconds to horizontally scale under sudden user load during active fire emergencies without risking core system downtime.

3.1.10 Deployment

- Amazon Web Services

The AWS Free Tier offers a zero-cost cloud environment to deploy and integrate our multi-service architecture, allowing us to test critical components like web hosting, file storage and push notification brokers without any upfront investment. It also provides immediate access to enterprise-grade security tools, including free SSL/TLS certificates and identity access management, letting us fully validate our system's functional capabilities before scaling up to paid infrastructure

3.1.11 Testing

- PyTest
- Playwright

Pytest allows us to rigorously validate the backend API, server-side role checks, and complex simulation edge-cases using reusable fixtures and parameterized data inputs. Meanwhile, Playwright drives actual browsers to emulate real-world field conditions, ensuring responsive map layouts, touch-drawn containment lines, and offline-caching layers work flawlessly for emergency responders.

4 API Contracts

4.1 Authentication & Access Control Service

Description: Manages user registration, login, and secure session management using JWT tokens.

Inputs:

- **email** (string) − User's unique email address.
- **password** (string) − User's secure password.

Outputs:

- **access_token** (JWT) − Token used for all subsequent authenticated API calls.
- **role** (string) − The assigned user role (e.g., Registered User, Firefighter, Admin).

Usage / Interaction Rules:

- Accessed via HTTPS/TLS.
- Tokens are invalidated immediately upon logout or role change.
- Authentication events are written to an audit log.

4.2 Fire Reporting & Verification Service

Description: Processes new fire reports from users and manages the verification status of incidents.

Inputs:

- **location** (coordinates) − GPS pin location of the fire.
- **timestamp** (DateTime) − Time the report is submitted.
- **fire_size** (string/int) − Estimated size of the fire.
- **photo** (file, optional) − Image attachment of the fire.
- **description** (string, optional) − Free-text details provided by the reporter.

Outputs:

- **reference_number** (string) − TA unique identifier for the report.
- **is_duplicate** (boolean) − Returns true if a report is a duplicate.
- **status** (string) − Initial status set to "Pending".

Usage / Interaction Rules:

- Can be called by anonymous (unregistered) users.
- Duplicate reports at the same location are automatically linked to one incident.

4.3 AI Fire Spread Simulation Engine

Description: Calculates predicted fire spread horizons based on environmental data and terrain.

Inputs:

- **fire_location** (coordinates) − The verified origin of the fire.
- **weather_data** (object) − Includes wind speed, direction, temperature, and humidity.
- **terrain_data** (object) − Includes slope and vegetation dryness index.
- **barriers** (array, optional) − Coordinates of logged containment lines.

Outputs:

- **spread_grid** (JSON/GeoJSON) − Colourcoded probability grid for 1h, 3h, 6h, and 24h horizons.

Usage / Interaction Rules:

- Triggered automatically once a fire status changes to "Verified".
- Re-runs automatically when new weather data or containment lines are received.

4.4 Live Fire Map Service

Description: Provides real-time data for fire markers and spread overlays to the client application.

Inputs:

- **user_role** (string) − Used to determine if the requester can see the spread overlay.
- **map_bounds** (coordinates) − The current viewport to filter visible markers.

Outputs:

- **active_fires** (list) − Array of verified fire markers with status, time, and size.
- **overlay_data** (object) − Prediction grid (visible to Registered Users only).

Usage / Interaction Rules:

- Service must support auto-refresh to push new data without manual reloads.
- Pending fires are filtered out and not shown to the public.

4.5 Proximity Alert Service

Description: Dispatches push notifications to users within a specific risk zone of a verified fire.

Inputs:

- **fire_id** (string) − Reference to the verified fire incident.
- **zone_id** (string) − The grid zone identified as at-risk.

Outputs:

- **delivery_status** (boolean) − Confirmation that the message was sent to the broker.

Usage / Interaction Rules:

- Must dispatch notifications within 5 seconds of verification.
- Messages must include a deep-link to the specific fire on the map.

4.6 Firefighter Tactical Service

Description: Handles specialized inputs from field personnel, such as GPS tracking and containment logging.

Inputs:

- **firefighter_id** (string) − Unique ID of the responder.
- **current_gps** (coordinates) − Real-time location of the firefighter.

- **containment_line** (array) – Set of GPS coordinates representing a firebreak.

Outputs:

- **sync_status** (string) – Confirmation of data storage and simulation trigger status.

Usage / Interaction Rules:

- Strictly restricted to users with the "Firefighter" role.
- Every logged action must be stored with a high-precision timestamp for auditing.

5 Deployment

5.1 Deployment Requirements

5.1.1 Live, Accessible System

Fire Away is accessible at: https://fireaway.csml.co.za

5.1.2 Environment Parity

- **Local Development:** All new features are first developed in our local environment in a Docker container.
- **Staging:** Code is pulled into the staging environment once all CI tests pass in local development. This environment is to remain stable for production.
- **Production:** This environment is stable and is the public release of the app.

5.1.3 Rollback Strategy

Our system infrastructure uses a Blue-Green Deployment rollback strategy that is backed by Caddy as a dynamic reverse proxy. Traffic is directed to one of two identical environments (slots blue or green) while the other remains idle.

Rollback Procedure

1. Re-initialize the previous slot: Start the stopped containers on the target rollback slot.
2. Network Re-attachment: Reconnect Caddy to the previous slot's network.
3. Proxy Swap: Swap Caddy's configuration back to the previous upstream target and issue a reload.
4. Isolate Failure: Stop the failing current slot for debugging.

5.2 Deployment Diagram

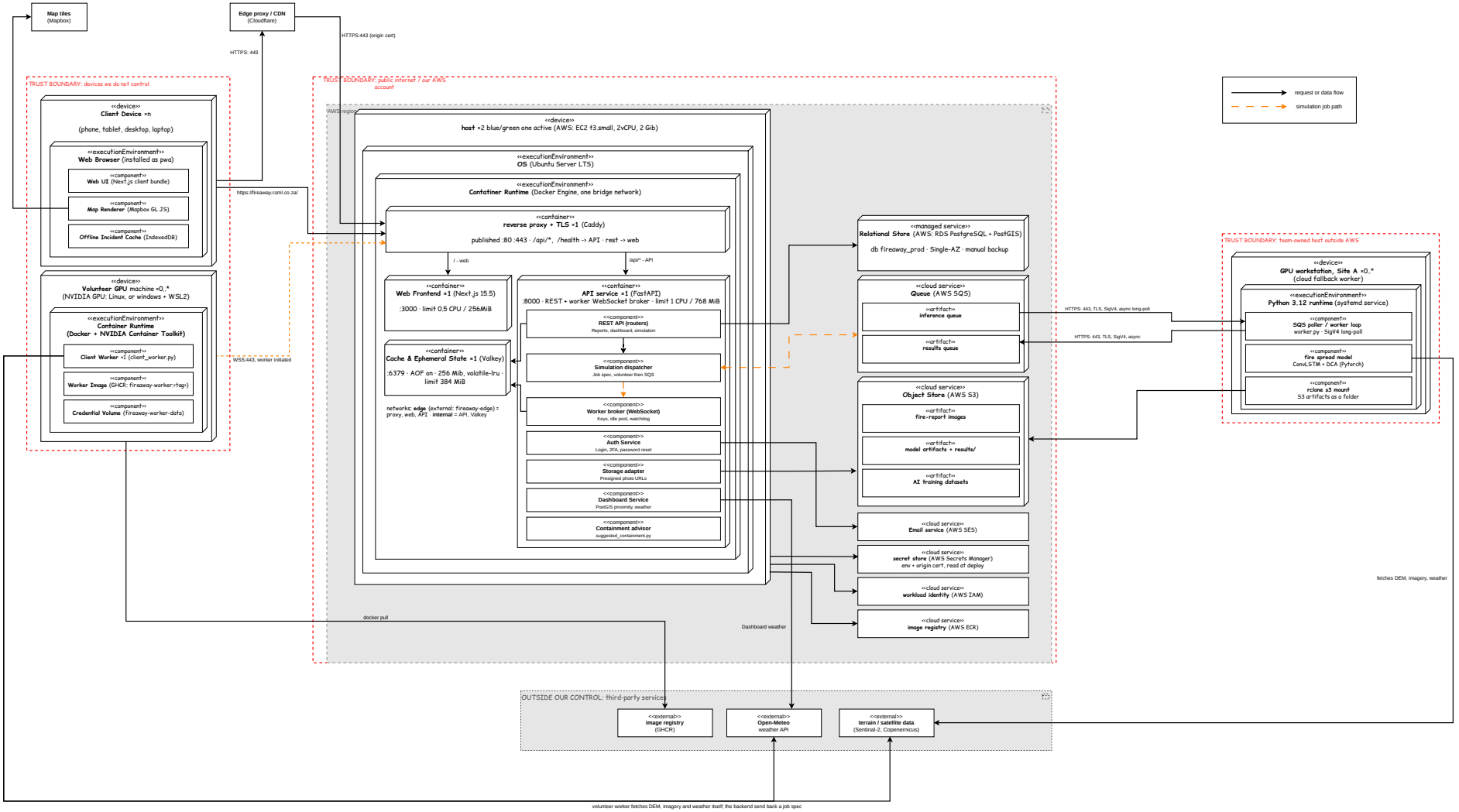


Figure 2: Deployment Diagram

5.3 CI/CD Pipeline Diagram

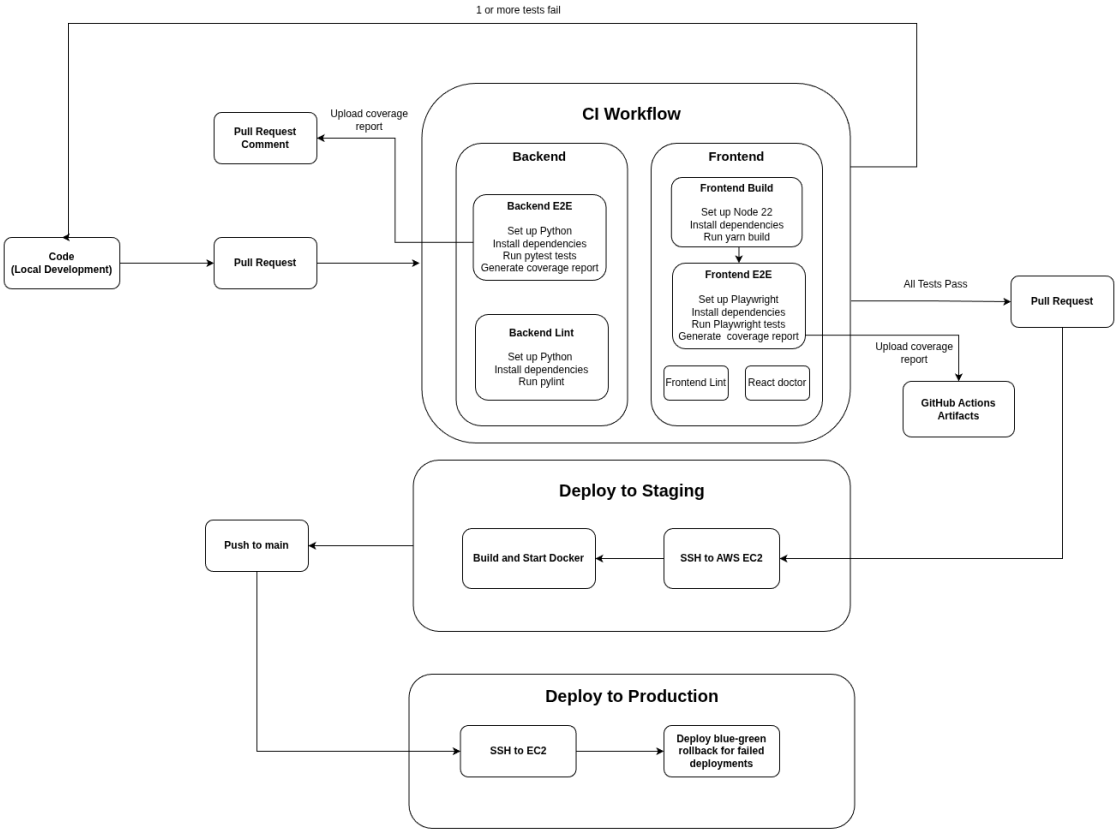


Figure 3: CI/CD Pipeline Diagram

6 Change Log

Date	Version	Change
July	1.0	Initial version
August	2.0	Architectural Requirements
October	3.0	Architecture Diagram, Patterns, Deployment